MODULE 30 · DOMAIN 4 - OPERATING, MONITORING, SECURING

Monitoring FMs and agents

Invocation metrics, prompt tracing, guardrail signals and agent trajectories.

MODULE 30 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

There is no label arriving next week to tell you the answer was wrong.

Failure modes a classical monitor would miss

- **Fluent and wrong.** The response is well formed, on topic, and factually incorrect.
- **Truncated output.** A streaming response stops mid-sentence and the client shows it anyway.
- **Over-refusal.** The model declines reasonable requests, which looks like success in every metric.
- **Silent retrieval failure.** Nothing relevant was found, and the model answered from training data.
- **Agent coordination failure.** The wrong tool, bad arguments, or a loop that never terminates.

> ON THE EXAM
>
> Every one of these returns HTTP 200. Standard application monitoring reports a perfectly healthy system.

CloudWatch generative AI observability

- A named CloudWatch capability for generative workloads, with pre-configured views.
- Monitors **model invocations, agents, knowledge bases, guardrails and tools** as distinct components.
- Provides **end-to-end prompt tracing**, so one request can be followed across components.
- Compatible with **Strands, LangChain and LangGraph**, so it is not tied to one framework.
- Third-party model traces go in through the **ADOT SDK**, from module 27.

> ON THE EXAM
>
> The component breakdown is the useful part. 'The answer was bad' becomes 'retrieval returned nothing and the model improvised'.

Model invocation metrics

METRIC	WHAT IT COVERS
Invocation count	Converse, ConverseStream, InvokeModel, and the streaming variant
Invocation latency	How long the call took
Token counts by model	Input and output tokens, delineated
Daily token counts by model ID	Where your spend is accumulating
Errors and throttling	Failures, and quota exhaustion. Module 25

ON THE EXAM

Token counts by model are a cost dashboard as much as a usage one.

A dashboard worth having

PANEL	WHY IT EARNS ITS PLACE
Invocations and errors	Is the system being used, and is it failing?
TimeToFirstToken, at p50 and p99	What users actually perceive on a streaming response
Token counts, input and output	Cost, and context growth over time
Throttles and EstimatedTPMQuotaUsage	How close you are to a wall
Guardrail interventions by policy	Whether safety is firing, and on what
Scheduled evaluation scores over time	The only quality line on the board

ON THE EXAM

Five operational panels and one quality panel. Most generative dashboards have the five and not the sixth.

Logs Insights for a specific complaint

- **Logs Insights** queries invocation logs, so you can find one request among millions.
- Filter by **time, model, and any identifier you logged**, then read the prompt and response.
- Aggregate too: count stop reasons, or find the longest prompts, or the most frequent refusals.
- Log a **request identifier your application controls**, so a support ticket maps to a log entry.
- Without that identifier, finding one complaint in production logs is guesswork.

> ON THE EXAM
>
> The request identifier is the cheapest thing on this slide and the one that makes every future investigation possible.

Model invocation logging is off by default

- **Metrics show that an invocation happened. Logging shows what was actually said.**
- **Model invocation logging is disabled by default** and must be enabled in Bedrock settings.
- Destination is CloudWatch Logs, or both S3 and CloudWatch Logs.
- Without it, you cannot investigate a bad answer, because there is no record of the prompt.
- Logged prompts and responses contain **whatever users typed**. Module 08 applies, and masking matters.

> ON THE EXAM
>
> This is the same shape as module 20's data capture: off by default, cannot be applied retrospectively, and needed the first time something goes wrong.

Sensitive data in logs

- Enabling invocation logging captures **whatever users typed**, including things they should not have.
- CloudWatch offers **sensitive data masking** on log groups. Apply it when you enable logging, not later.
- Restrict who can read the log group. It is now one of the most sensitive stores you have.
- Set a **retention period** deliberately. Indefinite retention of user prompts is a liability.
- Module 08's erasure obligations reach these logs too.

> ON THE EXAM
>
> Turning on logging solves an observability problem and creates a data protection one. Both need handling in the same change.

Sampling and retention

- At high volume, logging every invocation is expensive in ingestion and storage.
- **Sample** for routine observability, and keep **100 percent for anything flagged** or escalated.
- Metrics are cheap and complete. Logs are expensive and detailed. Use each for what it is good at.
- Sampling means the one request you need may not be there. Weigh that against the bill.
- Longer retention for a regulated workload, shorter for an internal tool. Decide, do not default.

> ON THE EXAM
>
> Same trade as module 20's data capture sampling: complete records cost money, and partial records fail you exactly once.

TimeToFirstToken and EstimatedTPMQuotaUsage

- **TimeToFirstToken** measures request sent to first token received, for the **streaming APIs**.
- You can alarm on it and set SLA baselines **without any client-side instrumentation**.
- That matters because perceived latency for a streaming response is time to first token, not total time.
- **EstimatedTPMQuotaUsage** gives visibility into how much of your token quota you are consuming.
- So you can see quota exhaustion approaching, rather than discovering it through throttling.

> ON THE EXAM
>
> Both were announced in March 2026, so most material predates them. They close two gaps people previously instrumented by hand.

Truncated and incomplete responses

- A streaming response can stop early: **maxTokens reached**, a timeout, or a dropped connection.
- **Check the stop reason**, from module 16. 'max_tokens' means the answer was cut off, not finished.
- Clients frequently display a truncated answer as if it were complete.
- Count truncations as a metric. A rising rate means maxTokens is set too low for real requests.
- Output token counts near your maxTokens ceiling are the same signal, seen from the metrics side.

> ON THE EXAM
>
> Module 16 introduced the stop reason as a field to read. This is what happens when nobody reads it in production.

Guardrail runtime metrics

METRIC	WHAT IT TELLS YOU
InvocationsIntervened	How often the guardrail actually blocked or modified something
Invocations	Requests to the ApplyGuardrail operation
InvocationLatency	What the guardrail costs you in response time
TextUnitCount	Text units consumed by the guardrail policies. The cost driver
InvocationThrottles	Throttled requests. They count as neither invocations nor errors
InvocationClientErrors and ServerErrors	Failures on either side

ON THE EXAM

InvocationsIntervened is the one to watch. It is the only metric that tells you the guardrail is doing anything at all.

The dimensions that make interventions useful

- **GuardrailContentSource**: was it the **Input** or the **Output** that triggered it?
- **GuardrailPolicyType**: content, topic, word, or sensitive information policy.
- Together these turn 'interventions rose' into 'the topic policy is firing on input twice as often'.
- **A rising intervention rate is ambiguous.** Either users changed, or your policy is too broad.
- Interventions on **output** are the more alarming direction: the model produced something blocked.

> ON THE EXAM
>
> Input interventions tell you about your users. Output interventions tell you about your model. They are different problems.

Watching for injection and misuse

- **Prompt injection attempts** often show as guardrail interventions on **input**, clustered by user.
- Unusual prompt length, repeated near-identical prompts, or rapid retries after a refusal.
- For an agent, watch for **tool calls that do not match the stated task**. Module 27's threat.
- Retrieved content containing instruction-shaped text is worth flagging, not just serving.
- Rate-limit per user, not only per account, or one caller can consume the whole quota.

> ON THE EXAM
>
> An agent that suddenly calls an unexpected tool is the signal that matters most, because tool calls have consequences.

Anomaly detection on usage patterns

- **CloudWatch anomaly detection** learns a metric's normal band and alarms on departures from it.
- Useful where a fixed threshold is wrong: traffic that varies by time of day and day of week.
- Good candidates: tokens per request, refusal rate, steps per task, retrieval result counts.
- It detects change, not badness. A legitimate product launch looks exactly like an anomaly.
- Pair it with a static alarm on the things that are always bad, like errors and throttling.

> ON THE EXAM
>
> Static thresholds suit metrics with a hard limit. Anomaly detection suits metrics where 'normal' depends on the hour.

End-to-end prompt tracing

- A trace follows one request through retrieval, tools, model calls and guardrails.
- So a bad answer becomes a **sequence you can read**, rather than a single disappointing output.
- It shows which chunks were retrieved, which tools were called, and how long each stage took.
- That is how you tell a retrieval problem from a generation problem, which module 18 needed.
- It needs enabling, and for third-party models it needs ADOT instrumentation.

> ON THE EXAM
>
> Module 18 told you retrieval and generation fail differently. Tracing is how you tell which one failed on a specific request.

Turning on AgentCore observability

- Built-in metrics for runtime, memory, gateway, tools and identity arrive in CloudWatch by default.
- To see **spans and traces**, complete the one-time **CloudWatch Transaction Search** setup.
- For the full range, and custom runtime metrics, instrument with the **ADOT SDK**.
- **Tracing must be enabled when a memory is created** to get service-provided spans for it.
- Module 27 introduced these. This is the point at which not having done it hurts.

> ON THE EXAM
>
> Two of these cannot be applied retrospectively. An agent already in production without them is an agent you cannot debug.

What to watch on a live knowledge base

SIGNAL	WHAT IT MEANS
Retrieval returning zero or few chunks	Queries outside the corpus, or a filter too narrow
Rising retrieval latency	Vector store pressure, or reranking added
Sync recency	How stale the index is. Module 10's deletion risk
Ingestion job failures	A silently failing subset of documents. Module 26
Retrieved tokens per request	Cost, and context window pressure
Citation rate	Answers produced with no citation are answers from training data

ON THE EXAM

The last row is the cheap proxy for faithfulness: an uncited answer in a grounded system is a warning without needing a judge model.

What to watch on a live agent

SIGNAL	WHAT IT MEANS
Steps per task	The leading indicator. Rising means the agent is struggling
Tasks hitting the step limit	The runaway rate. Should be near zero
Tool error rate, per tool	One failing tool degrades every task that needs it
Tool selection distribution	A tool suddenly chosen far more or less often
Escalation rate to humans	What the agent cannot do, reported by the agent
Task completion rate	The outcome measure, where you can define completion

ON THE EXAM

Steps per task rises before quality falls and before the bill arrives. It is the earliest warning an agent gives you.

Cost monitoring per feature and tenant

- Token counts by model give an account total. They do not say which feature spent it.
- **requestMetadata** from module 16 tags invocations, so spend can be attributed afterwards.
- Or count tokens in your application, per tenant and per feature, and emit a custom metric.
- Alarm on **tokens per request**, not just total tokens. A rising average means prompts are growing.
- An agentic feature can multiply spend without any change in user numbers. Module 27.

> ON THE EXAM
>
> Tokens per request is the diagnostic. Total tokens rising with traffic is growth; tokens per request rising is a problem.

Coordination failures

- **Handoff loops:** two agents passing work back and forth without progressing.
- **Duplicated work:** several agents doing the same thing because the split was unclear.
- **Dropped context:** a specialist agent receives a task without what it needs to do it.
- These show as **rising steps with a falling completion rate**, which is the signature to look for.
- Module 27's gap makes this more likely: much of the coordination is custom code you wrote.

> ON THE EXAM
>
> The trace is the only way to see these. An aggregate metric shows a slow expensive system without saying why.

Monitoring the knowledge base refresh

- An **ingestion job** runs on a schedule. Monitor that it ran, and that it succeeded.
- **Per-document failures are the quiet risk.** A job can succeed while 6 percent of documents failed.
- Alarm on **failed document count**, not only on job status.
- Track **sync recency**, because module 10's deleted document stays retrievable until the next sync.
- A sudden change in document count usually means a source system changed, not the pipeline.

> ON THE EXAM
>
> A knowledge base degrades silently: nothing errors, answers just quietly stop covering the newest material.

Scheduled evaluation in production

- Run module 18's evaluation set on a schedule against the live configuration.
- **This is how you get a quality number at all**, since production supplies no labels.
- Watch the **change between runs**, not the absolute value. A drop is the signal.
- Per metric: falling faithfulness and falling context relevance mean different things. Module 18.
- Model versions move, documents change, prompts get edited. All of them degrade quality silently.

> ON THE EXAM
>
> This is the direct equivalent of module 29's drift monitoring, for a system where accuracy never arrives on its own.

Detecting what changed underneath you

- A **model version moved**, and nothing in your code changed. Module 16's pinning point.
- A **prompt was edited**, which is a production change with no deployment.
- **Source documents changed**, so retrieval returns different passages for the same question.
- Record the **model ID, prompt version and index sync time** alongside each evaluation run.
- Then a quality drop can be lined up against what changed, instead of investigated blind.

> ON THE EXAM
>
> Three things can change your system without a deployment. Recording all three is what turns 'quality fell' into 'quality fell when X changed'.

Synthetic checks

- Send a small set of **known questions on a schedule**, and check the answers automatically.
- It catches a broken system at 4am, rather than when the first user arrives at 9.
- Assert on things that are checkable: a citation present, a keyword present, a plausible length.
- Cheaper and faster than a full evaluation, so it can run far more often.
- It tests availability and basic sanity. It does not replace scheduled evaluation.

> ON THE EXAM
>
> A synthetic check is a smoke test: it will not tell you quality declined, and it will tell you retrieval has been returning nothing since midnight.

User feedback

- A thumbs up and down is the cheapest quality signal available, and it is real ground truth.
- **Negative feedback with its trace attached** is the single most useful artefact for debugging.
- Feed those cases into the evaluation set, from module 18's maintenance slide.
- Feedback is biased: people rate when annoyed. Treat the rate as a trend, not an accuracy estimate.
- Implicit signals count too: did they rephrase, retry, or abandon?

> ON THE EXAM
>
> Users are the only source of real labels in a generative system, and they give them away for the price of a button.

Monitoring the outcome, not the output

- The strongest signal is whatever the system was built to change: deflection, resolution, time saved.
- **Those are real outcomes**, and they are the closest thing to accuracy you will get in production.
- Escalation to a human is a business metric and a quality metric at the same time.
- Repeat questions from the same user suggest the first answer did not land.
- They are slow and confounded. Watch them alongside proxies, not instead of them.

> ON THE EXAM
>
> Module 17's rule arrives in Domain 4: the right metric is the one that moves when the business outcome moves.

Running two configurations at once

- Route a share of traffic to a second configuration: a different model, prompt, or top-k.
- Compare **feedback, escalation and outcome metrics** between them on the same real traffic.
- Bedrock has no deployment guardrails, from module 25, so your application does the routing.
- Change one thing at a time, so the difference is attributable. Module 15's discipline.
- Run it long enough for the difference to mean something. Module 17's sample-size point.

> ON THE EXAM
>
> This is module 24's A/B test, rebuilt in the application layer because the managed version does not exist for Bedrock.

What to alarm on, and what to do

- Set **warning and error thresholds** from your workload's expected patterns, not from defaults.
- Alarm on: **throttling, error rate, TimeToFirstToken, step limit hits, ingestion failures**.
- The Well-Architected guidance is explicit: **define and practise incident response playbooks**.
- An alarm with no playbook produces a person at 3am reading documentation.
- Composite alarms reduce noise when several related signals move together.

> ON THE EXAM
>
> Practising the playbook is the part everyone skips, and it is the part that decides how the incident actually goes.

SLOs for a generative system

- Availability and latency work normally: **error rate** and **TimeToFirstToken at p95**.
- A quality SLO is harder, because the measurement is periodic rather than continuous.
- A workable form: **evaluation scores stay above a threshold on every scheduled run**.
- Add a behavioural one where it matters: refusal rate, or escalation rate, within a band.
- An SLO you cannot measure is a wish. Only commit to what your telemetry actually supports.

> ON THE EXAM
>
> Two of the three classic SLO dimensions transfer directly. The third has to be redefined around scheduled measurement.

What a playbook should contain

- **What the alarm means**, in one sentence somebody can read at 3am.
- **The first three things to check**, in order, with links to the right dashboard or query.
- **How to roll back**: the previous model ID, the previous prompt version, the previous configuration.
- Who to escalate to, and when to stop investigating and mitigate instead.
- AWS says to **practise** these. A playbook first read during an incident is a document, not a plan.

> ON THE EXAM
>
> Module 24's rollback practice again, in a different domain. Knowing the rollback takes four minutes is worth more than knowing it exists.

Things that surprise people

BEHAVIOUR	CONSEQUENCE
Metrics are aggregated datapoints	They do not map to single invocations. Use logs
Log groups are per Region	Create subscriptions and streams in each Region
Providers emit different schemas	Parsing for one model may not fit another
Logging is off by default	Nothing is recorded, and not retrospectively
Throttles are not errors	An error-rate alarm will not catch throttling

ON THE EXAM

The last row matters: throttling needs its own alarm.

Multi-Region and multi-account

- **Log groups and metrics are per Region.** A cross-Region inference profile produces telemetry in several.
- Subscriptions and metric streams must be created **in each Region** you invoke from.
- **CloudWatch cross-account observability** aggregates into a monitoring account.
- That matters with module 28's multi-account structure, where production is its own account.
- A dashboard built in one Region will silently omit traffic from the others.

> ON THE EXAM
>
> Module 25's cross-Region inference profile improves throughput and scatters your telemetry. Both are true.

Symptom to signal

WHAT YOU NEED TO KNOW	WHERE TO LOOK
Are responses getting slower to start?	TimeToFirstToken
Are we near a quota?	EstimatedTPMQuotaUsage, and throttling
What exactly did the model see and say?	Model invocation logs. Enable them first
Is the guardrail doing anything?	InvocationsIntervened, by source and policy type
Why was this one answer bad?	End-to-end prompt trace
Is quality degrading overall?	Scheduled evaluation, run to run
Is the agent struggling?	Steps per task, and step limit hits
Are users unhappy?	Feedback rate, and retries

ON THE EXAM

Note the split: metrics answer 'is something wrong', traces answer 'what happened here', evaluation answers 'is it still good'.

The minimum monitoring to ship with

SET UP BEFORE LAUNCH	WHY IT CANNOT WAIT
Model invocation logging, with masking	Cannot be enabled retrospectively
A request identifier in every call	Otherwise complaints cannot be traced
Alarms on errors, throttling and TimeToFirstToken	The three that page you
A frozen evaluation set, running on a schedule	The only quality baseline you will have
A feedback control in the interface	The only real labels you will ever get
Tracing enabled, including on memories	Two parts cannot be turned on later

ON THE EXAM

Four of these six become impossible or useless if added after an incident. That is the argument for doing them on day one.

What monitoring itself costs

- **CloudWatch metrics** are cheap. Custom metrics cost per metric, so do not emit thousands.
- **Log ingestion and storage** dominate, and invocation logs are large because prompts are large.
- **Traces** add ingestion cost, and Transaction Search has its own charge.
- **Scheduled evaluation** costs model calls every run. A large set run hourly is a real line item.
- Size the evaluation set and the sampling rate deliberately. Both are dials, not defaults.

> ON THE EXAM
>
> Monitoring a generative system costs meaningfully more than monitoring a classical one, because the artefacts are text and the quality check is a model call.

What not to alarm on

- **Every guardrail intervention.** Interventions are the guardrail working. Alarm on the rate changing.
- **Individual slow requests.** Alarm on a percentile over a window, not on one outlier.
- **Any single evaluation metric moving slightly.** Run-to-run variance is real, from module 18.
- **Successful pipeline runs and successful syncs.** Success is the expected case.
- Every alarm that fires without needing action makes the next real one less likely to be read.

> ON THE EXAM
>
> Alert fatigue is a monitoring failure, not a people failure. A channel people mute is worse than no channel.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Cannot investigate a complaint	Invocation logging was never enabled
Answers cut off mid-sentence	maxTokens reached. Check the stop reason
Dashboard healthy, users unhappy	Everything returns 200. You are measuring the wrong things
Guardrail interventions spiked	Check the dimensions: input or output, and which policy
Agent cost rose with no more users	Steps per task rising. Something changed upstream
Error alarm never fires during throttling	Throttles are neither invocations nor errors
Quality fell with no deployment	A model version moved, or a prompt or document changed

ON THE EXAM

Row three is the characteristic failure of this domain. A green dashboard is not evidence that a generative system is working.

Classical monitoring against generative monitoring

QUESTION	CLASSICAL MODEL (M29)	GENERATIVE SYSTEM (M30)
Ground truth	Arrives later. Accuracy computable	Never arrives. Manufacture it
Primary signal	Data drift, then model quality	Proxies, plus scheduled evaluation
A failure is	A metric crossing a threshold	A 200 response that is wrong
Investigation unit	A prediction and its features	A trace across components
The tooling	Model Monitor, baselines, Clarify	GenAI observability, evaluations

ON THE EXAM

Row one causes every other difference in this table.

Reporting to people who are not engineers

- Operational metrics mean nothing to a product owner. **Translate into cases.**
- 'Refusal rate rose from 2 to 6 percent' means **about 40 more users a day were declined**.
- Report the **worst segment**, not just the average. Module 17's point, in production.
- Show the evaluation trend over time. It is the only quality line stakeholders will understand.
- A dashboard nobody outside the team reads will not survive a budget conversation.

ON THE EXAM

Module 17's reporting slide applies here too: a number with an interpretation is actionable, and a bare number is not.

Watch the proxies, and schedule the evaluation. Production will not tell you the answer was wrong.

> Users of a streaming chat assistant complain that it 'takes ages to start responding', though total response times look normal on the existing dashboard.
>
> The team wants to alarm on this and set an SLA, but the application is a third-party client they cannot instrument.

What should they use?

A Invocation latency, alarming when it exceeds a threshold

B Model invocation logs, timing the gap between request and response entries

C The TimeToFirstToken CloudWatch metric

D A synthetic canary calling the model every minute and timing it

◆ PAUSE AND DECIDE

What should they use?

A. Invocation latency, alarming when it exceeds a threshold

Measures total call duration, which the scenario says looks normal. For a streaming response that is not what users perceive.

B. Model invocation logs, timing the gap between request and response entries

Logs record the invocation, not the moment the first token arrived. It also means building metric extraction from log parsing.

C. The TimeToFirstToken CloudWatch metric

Measures exactly request-sent to first-token-received on the streaming APIs, and AWS states it needs no client-side instrumentation. Alarms and SLA baselines are its stated purpose.

D. A synthetic canary calling the model every minute and timing it

Measures a synthetic request rather than real traffic, and builds infrastructure to reproduce a metric that already exists.

THE SIGNAL

The constraint 'cannot instrument the client' is the deciding one, and it is exactly what the March 2026 metric was released for.

> A customer escalates a specific answer from a Bedrock-backed assistant as factually wrong and potentially harmful. Compliance asks for the exact prompt the model received and the exact response it produced.
>
> The team has CloudWatch metrics dashboards showing invocation counts, latency and token usage. They cannot produce the prompt or the response.

Why, and what should change?

A. Model invocation logging was never enabled. Enable it, with sensitive data masking

B. CloudWatch retention expired. Increase the retention period

C. Metrics cannot be filtered per user. Add a user dimension to the metrics

D. Bedrock does not retain prompts. Log them in the application instead

◆ PAUSE AND DECIDE

Why, and what should change?

A Model invocation logging was never enabled. Enable it, with sensitive data masking

Metrics show that invocations happened; logging shows their content, and it is off by default. Masking addresses the module 08 obligation the logs create.

B CloudWatch retention expired. Increase the retention period

Retention cannot expire logs that were never written. Nothing was captured in the first place.

C Metrics cannot be filtered per user. Add a user dimension to the metrics

Metrics are aggregated counts and never contain prompt or response text, however they are dimensioned.

D Bedrock does not retain prompts. Log them in the application instead

Bedrock does offer invocation logging. Application-side logging is a reasonable supplement and not the answer to a feature that exists and was left off.

THE SIGNAL

The same shape as module 20's data capture: off by default, and it cannot be turned on retrospectively for an incident that already happened.

> Guardrail interventions on an internal assistant have tripled over two weeks. Support reports users complaining that ordinary questions are being refused. Nothing was deployed in that period.
>
> The team can see the total InvocationsIntervened count rising, and nothing else.

What should they look at next?

A. Model invocation latency, to see whether the guardrail is the bottleneck

B. TextUnitCount, to see whether guardrail cost has risen proportionally

C. Total invocation count, to see whether traffic simply grew

D. InvocationsIntervened split by GuardrailContentSource and GuardrailPolicyType

◆ PAUSE AND DECIDE

What should they look at next?

A Model invocation latency, to see whether the guardrail is the bottleneck

Answers a performance question. The complaint is about refusals, which is a behaviour question.

B TextUnitCount, to see whether guardrail cost has risen proportionally

Useful for cost, and it rises with traffic regardless of interventions. It does not say which policy is firing.

C Total invocation count, to see whether traffic simply grew

A sensible sanity check, and it cannot explain refusals of ordinary questions. It distinguishes rate from volume, no more.

D InvocationsIntervened split by GuardrailContentSource and GuardrailPolicyType

The dimensions say whether it fired on input or output, and which policy. That turns an ambiguous rise into a specific, fixable cause.

THE SIGNAL

A rising intervention count has two opposite explanations. The dimensions are the only thing that separates 'users changed' from 'policy too broad'.

> A director asks the team to 'put an accuracy alarm on the assistant, like we have on the fraud model'. The fraud model's accuracy is computed nightly against outcomes that arrive within 48 hours.
>
> The assistant answers policy questions. No outcome data is ever generated for its answers.

How should you respond?

A. Agree, and compute accuracy from the guardrail intervention rate

B. Explain there is no label, and propose scheduled evaluation plus proxy alarms and feedback

C. Agree, and use invocation errors as the accuracy proxy

D. Explain that accuracy cannot be measured for foundation models at all

◆ PAUSE AND DECIDE

How should you respond?

A Agree, and compute accuracy from the guardrail intervention rate

Interventions measure policy violations, not correctness. A confidently wrong policy answer passes every guardrail.

B Explain there is no label, and propose scheduled evaluation plus proxy alarms and feedback

Scheduled evaluation creates ground truth deliberately, proxies alarm on degradation, and feedback supplies real labels. It delivers what was asked for by a different route.

C Agree, and use invocation errors as the accuracy proxy

Errors mean the call failed. Every failure in this module returns HTTP 200, so an error rate says nothing about correctness.

D Explain that accuracy cannot be measured for foundation models at all

Too absolute, and unhelpful. Module 18 measures quality perfectly well; it just requires an evaluation set rather than production labels.

THE SIGNAL

The director's request is reasonable and the mechanism does not exist. The answer is to supply the outcome, not to refuse the question.

What to take from this module

- **Every failure here returns HTTP 200.** A green dashboard proves nothing about quality.
- **Model invocation logging is off by default**, and cannot be enabled retrospectively.
- **TimeToFirstToken** for streaming latency, with no client instrumentation. New in March 2026.
- **InvocationsIntervened**, split by content source and policy type, is the guardrail signal.
- **Schedule the evaluation.** It is the only quality measurement production will not give you.

> ON THE EXAM
>
> Module 31 widens this to observability across the whole ML platform: logs, metrics, traces and cost.